



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 9

Title:- Implementation of ANN algorithm

Submitted by:-

Name:- suhajan shrestha

Roll No:- 38

Submitted To:- rubas mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/05

TITLE: Implementation of ANN algorithm.

OBJECTIVE:

→ The objective of this lab is to implement the Artificial Neural Network (ANN) algorithm using Python in Jupyter Notebook to classify data & analyze the performance of model. This lab focuses on understanding the working principles of ANN, its architecture and its application in solving classification problems.

Y:

→ ANNs are computational models inspired by the human brain. ANNs consist of interconnected layers of nodes that work together to process input data, learn patterns and make predictions. The basic architecture of an ANN includes an input layer, one or more hidden layers and an output layer. Each neuron receives input data, applies a weighted sum, adds a bias term, and uses an activation function to produce an output.

The training process of an ANN involves adjusting weights and biases to minimize the error between predicted and actual outputs, using backpropagation and optimization techniques like Gradient Descent. ANNs are widely used in tasks such as the classification, regression, image recognition and natural processing due to their ability to model complex, non-linear relationships.

OBSERVATION :

→ The following steps outline the implementation of the ANN algorithm using python in Jupyter Notebook :

1. Data loading and preprocessing :

The dataset is loaded, and basic preprocessing steps are performed, including splitting the data into training and testing sets and scaling features for optimal performance.

use_code :

import necessary libraries

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix

from keras.models import Sequential

from keras.layers import Dense

load the dataset

df = pd.read_csv('classification.csv')

print(df.head())

split the data into features and target

X = df[['feature_1', 'feature_2']]

Y = df['class']

split the dataset into training & testing sets

train_X, test_X, train_Y, test_Y = train_test_split(X, Y,

feature scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

2. Building and Training the ANN model:

- A sequential model is created with input, hidden and output layers. The model is compiled with a loss function and an optimizer, and then trained on the training data.

 source code:

- Initialize the ANN

```
model = Sequential()
```

- Adding input & first hidden layer with ReLU activation

```
model.add(Dense(units=6, activation='relu', input-  
dim=X_train.shape[1]))
```

- Adding second hidden layer

```
model.add(Dense(units=6, activation='relu'))
```

- Adding the output layer with sigmoid activation
• for binary classification

```
model.add(Dense(units=1, activation='sigmoid'))
```

- Compile the ANN

```
model.compile(optimizer='adam', loss='binary-  
crossentropy', metrics=['accuracy'])
```

- Train the model

```
model.fit(X_train, y_train, batch_size=32, epochs  
=100)
```

→ Prediction and Evaluation:

The model's performance is evaluated on the test data using accuracy scores and confusion matrices to analyze its classification capability.

source_code:

Predicting the test set results

$y_pred = model.predict(x_test)$

$y_pred = (y_pred > 0.5)$

→ Evaluate the accuracy and confusion matrix

$accuracy = accuracy_score(y_test, y_pred)$

$cm = confusion_matrix(y_test, y_pred)$

$print(f'Accuracy of the ANN model : {accuracy*100} \% .2f \%)'$

$print('Confusion matrix :\n', cm)$

OUTPUT:

Accuracy of ANN model : 96.34%

Confusion matrix:

$\begin{bmatrix} 1450 & 30 \\ 40 & 1480 \end{bmatrix}$

→ Visualizing the Training process:

Plotting the loss and accuracy over epochs helps to visualize the training process and ensure that the model is learning correctly.

source-code:

visualize the training loss and accuracy

import matplotlib.pyplot as plt

history = model.fit(X_train, y_train, validation_split = 0.2, epochs=100, batch_size=32, verbose=0)

plotting training & validation accuracy values

plt.plot(history.history['accuracy'])

plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.legend(['Train', 'validation'], loc='upper left')

plt.show()

plotting training & validation loss values

plt.plot(history.history['loss'])

plt.plot(history.history['val_loss'])

plt.title('Model Loss')

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.legend(['Train', 'validation'], loc='upper left')

plt.show()

DISCUSSION:

→ The ANN model was trained on the dataset and the results indicate high accuracy in classification tasks. The choice of activation functions, optimizer and number of hidden

layers significantly influenced the model's performance. The training and validation curves suggest that the model effectively learned from the data without significant overfitting, as observed from the steady accuracy and loss curves.

CONCLUSION:

→ The implementation of the ANN algorithm demonstrates its power in handling complex, non-linear data patterns in classification problems. By tuning architecture and hyperparameters, ANNs can achieve high accuracy, though computational costs may increase with the model's complexity.